

```

#include <xc.h>

#include "timer.h"

#include "main.h"

#include "ADC.h"

#include "PWM.h"

#include "IO.h"

#include "ChipConfig.h"

unsigned char toggle = 0;

//Initialisation d'un timer 16 bits

void InitTimer1(void) {

    T1CONbits.TON = 0;

    T1CONbits.TCS = 0;

    SetFreqTimer1(50);    // ← ICI, ligne à corriger en SetFreqTimer1(50);

    IFS0bits.T1IF = 0;

    IEC0bits.T1IE = 1;

    T1CONbits.TON = 1;

}

//Interruption du timer 1

void __attribute__((interrupt, no_auto_psv)) _T1Interrupt(void) {
    ADC1StartConversionSequence();
    PWMUpdateSpeed();
    IFS0bits.T1IF = 0;
    //LED_BLANCHE_1 = !LED_BLANCHE_1;
}

//Initialisation d'un timer 32 bits

void InitTimer23(void) {
    T3CONbits.TON = 0; // Stop any 16-bit Timer3 operation
    T2CONbits.TON = 0; // Stop any 16/32-bit Timer3 operation
    T2CONbits.T32 = 1; // Enable 32-bit Timer mode
    T2CONbits.TCS = 0; // Select internal instruction cycle clock
    T2CONbits.TCKPS = 0b00; // Select 1:1 Prescaler
    TMR3 = 0x00; // Clear 32-bit Timer (msw)
    TMR2 = 0x00; // Clear 32-bit Timer (lsw)
    //PR3 = 0x0393; // Load 32-bit period value (msw)
    //PR2 = 0x8700; // Load 32-bit period value (lsw)
}

```

```

    PR3 = 0x0727;
    PR2 = 0x0E00;
    IPC2bits.T3IP = 0x01; // Set Timer3 Interrupt Priority Level
    IFS0bits.T3IF = 0; // Clear Timer3 Interrupt Flag
    IEC0bits.T3IE = 1; // Enable Timer3 interrupt
    T2CONbits.TON = 1; // Start 32-bit Timer
}
//Interruption du timer 32 bits sur 2-3

void __attribute__((interrupt, no_auto_psv)) _T3Interrupt(void) { IFS0bits.T3IF = 0; }

void SetFreqTimer1(float freq) {
    T1CONbits.TCKPS = 0b00;
    if (FCY / freq > 65535) {
        T1CONbits.TCKPS = 0b01;
        if (FCY / freq / 8 > 65535) {
            T1CONbits.TCKPS = 0b10;
            if (FCY / freq / 64 > 65535) {
                T1CONbits.TCKPS = 0b11;
                PR1 = (int) (FCY / freq / 256);
            } else
                PR1 = (int) (FCY / freq / 64);
        } else
            PR1 = (int) (FCY / freq / 8);
    } else
        PR1 = (int) (FCY / freq);
}

unsigned long timestamp = 0;

void SetFreqTimer4(float freq)
{
    T4CONbits.TCKPS = 0b00;
    if (FCY / freq > 65535) {
        T4CONbits.TCKPS = 0b01;
        if (FCY / freq / 8 > 65535) {
            T4CONbits.TCKPS = 0b10;
            if (FCY / freq / 64 > 65535) {
                T4CONbits.TCKPS = 0b11;
                PR4 = (int) (FCY / freq / 256);
            } else
                PR4 = (int) (FCY / freq / 64);
        } else
            PR4 = (int) (FCY / freq / 8);
    } else
        PR4 = (int) (FCY / freq);
}

void InitTimer4(void) {
    T4CONbits.TON = 0;
    T4CONbits.TCS = 0;
    SetFreqTimer4(1000); // 1kHz = incrémentation toutes les 1ms
    IFS1bits.T4IF = 0;
    IEC1bits.T4IE = 1;
}

```

```
        T4CONbits.TON = 1;
    }

    void __attribute__((interrupt, no_auto_psv)) _T4Interrupt(void) {
        IFS1bits.T4IF = 0;
        timestamp++;
        OperatingSystemLoop();
    }
```